

# nf2vec: Deep Learning on Object-centric Neural Fields

Hot Topics in Computer Vision Seminar

Technical University of Berlin

Winter Semester 2025/26

Oliver Jan Jarosik 0501933

jarosik@campus.tu-berlin.de

---

## Contents

---

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>2</b>  |
| <b>2</b> | <b>3D Representations and Neural Fields</b>         | <b>3</b>  |
| 2.1      | Neural Radiance Fields . . . . .                    | 3         |
| <b>3</b> | <b>The NF2Vec Framework</b>                         | <b>4</b>  |
| 3.1      | Pipeline Overview . . . . .                         | 4         |
| 3.2      | Architecture Design . . . . .                       | 4         |
| 3.3      | Training Methodology . . . . .                      | 5         |
| 3.4      | Downstream Applications . . . . .                   | 5         |
| 3.5      | Evaluation and Performance . . . . .                | 6         |
| <b>4</b> | <b>Applications and Extensions</b>                  | <b>7</b>  |
| 4.1      | Latent Space Interpolation . . . . .                | 7         |
| 4.2      | Cross-modal Completion and Reconstruction . . . . . | 8         |
| <b>5</b> | <b>Experiments</b>                                  | <b>9</b>  |
| 5.1      | Experimental Setup . . . . .                        | 9         |
| 5.2      | Embedding Quality Evaluation . . . . .              | 10        |
| 5.3      | Linear Mode Connectivity Analysis . . . . .         | 10        |
| 5.4      | Cross-Dataset Generalization . . . . .              | 11        |
| <b>6</b> | <b>Discussion and Limitations</b>                   | <b>12</b> |
|          | <b>References</b>                                   | <b>13</b> |
| <b>7</b> | <b>Appendix</b>                                     | <b>14</b> |

## Introduction

---

3D Computer Vision faces a challenge of representation. Unlike 2D images with uniform pixel grids, 3D data lacks a canonical structure, leading to competing formats. Voxel grids scale poorly, point clouds lose spatial continuity, and meshes struggle with complex topologies requiring researchers to develop specialized algorithms for each representation and hindering unified frameworks. There is no natural way to extract meaningful features from raw network parameters. Neural Fields (NFs) offer an alternative by encoding 3D objects as continuous functions parameterized by neural networks [1, 2, 3, 4]. This creates a paradox: NFs can theoretically unify 3D representations, but they cannot be directly integrated into existing deep learning pipelines that have proven successful for discrete formats. But using networks themselves as input to downstream tasks remains difficult, since traditional models expect structured arrays rather than large weight vectors. The nf2vec [5] framework tackles this by learning embeddings from neural field parameters. It processes weights in a permutation-aware fashion, using shared transformations and symmetric pooling, enabling standard deep learning pipelines to operate on neural representations. This work explores nf2vec’s design, training, and performance across tasks such as classification, retrieval, and cross-modal completion, highlighting its potential to bridge continuous 3D representations with discrete learning algorithms. Section 2 provides context by examining traditional 3D representations and their limitations. Section 3 presents the core nf2vec framework, detailing its architecture, training methodology, and performance evaluation. Section 4 explores practical applications and demonstrates the versatility of the approach, with particular emphasis on latent space interpolation capabilities. Section 5 presents experimental validation of the framework. Section 6 discusses current limitations and directions for future development.

## 3D Representations and Neural Fields

---

The choice of 3D representation shapes algorithmic possibilities and computational demands. Traditional discrete formats optimize for certain tasks but each comes with characteristic limitations.



**Figure 1:** Representation of Point Cloud, Voxel Grid, Mesh

**Voxel grids** divide 3D space into regular cubic cells, enabling the direct application of 3D convolutional networks [6]. Memory grows cubically with resolution, making high fidelity expensive. **Point clouds** represent geometry as unordered sets of 3D coordinates, sometimes with colors or normals. They capture surfaces efficiently, scaling with geometric complexity rather than volume, and arise naturally from depth sensors and LiDAR scanners [7, 8]. But the lack of ordering complicates local operations and requires tailored networks [9]. **Triangle meshes** encode topology through vertices and connectivity. They excel at smooth surfaces and rendering, adapt detail where needed, and remain efficient in simple areas [10]. Yet, holes and non-manifold structures are challenging, and subsurface detail is absent. Each format demands specialized models: 3D CNNs for voxels, permutation-equivariant networks for point clouds [7, 8, 9], and graph networks for meshes [10]. This specialization fragments research and hinders unified 3D understanding.

### 2.1 Neural Radiance Fields

Neural Radiance Fields (NeRFs) [1] exemplify how neural fields can unify 3D representation. A NeRF models a scene as a continuous volumetric function  $f : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ , mapping 3D positions  $\mathbf{x}$  and viewing directions  $\mathbf{d}$  to RGB colors  $\mathbf{c}$  and volume density  $\sigma$  through a multilayer perceptron. The elegance of NeRFs lies in their implicit representation: the entire scene exists within the network’s weights, queryable at any spatial resolution without additional memory cost. Training requires only posed 2D images, sidestepping the need for explicit 3D supervision. Through differentiable volume rendering, NeRFs learn to reconstruct complex geometries and view-dependent effects from image observations alone. Other neural field approaches include implicit neural representations with periodic activation functions [2], signed distance functions [3], and occupancy networks [4]. Each type targets specific tasks but shares the principle of function approximation through neural networks.

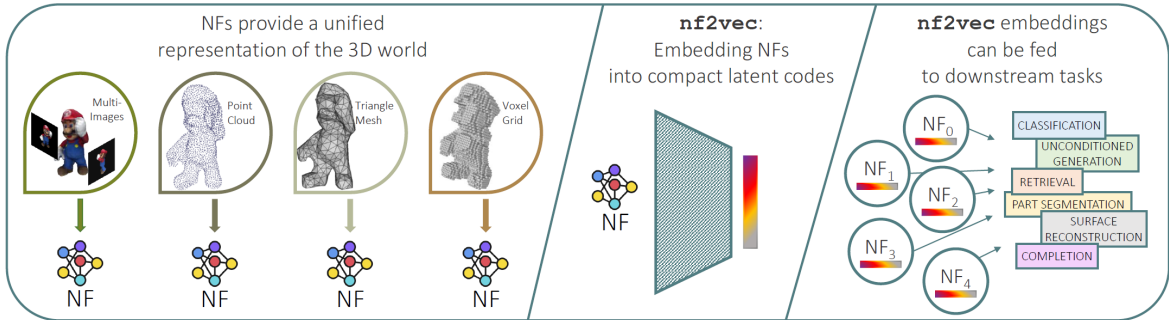
The success of neural fields prompts a central question: if 3D data can be universally represented as neural networks, can we perform deep learning directly on these network representations? The nf2vec framework [5] provides an affirmative answer by enabling standard machine learning pipelines to operate on neural field parameters.

## The NF2Vec Framework

The nf2vec framework [5] enables deep learning directly on neural field parameters by learning compact, semantically meaningful embeddings. Given a neural field  $f_\theta$  with parameters  $\theta$ , nf2vec produces an encoder  $E : \theta \mapsto \mathbf{z} \in \mathbb{R}^d$  that transforms network weights into fixed-dimensional vectors suitable for classification, retrieval, and manipulation tasks. The approach processes parameters in a row-wise transformation and symmetric aggregation, allowing standard machine learning architectures to operate on neural representations without explicit knowledge.

### 3.1 Pipeline Overview

The nf2vec pipeline operates in two phases: **training** and **inference**. During **training**, individual neural fields representing 3D objects are fitted to their respective data using standard optimization procedures, all initialized from the same random seed. These trained networks serve as input to the nf2vec encoder, which processes their weight parameters to produce compact embeddings. A lightweight decoder receives these embeddings along with spatial coordinates, predicting field values that are compared against ground truth through reconstruction losses. This encoder-decoder training ensures that embeddings preserve functionally relevant information.



**Figure 2:** Overview of the nf2vec pipeline from 3D representation to embeddings

At **inference**, the trained encoder processes novel neural fields through a single forward pass, generating embeddings that can be used for various downstream applications. This separation enables efficient deployment where encoding cost remains constant regardless of complexity. The core challenge addressed by nf2vec lies in the computational and semantic difficulties of processing network parameters directly. **Weight permutations** in hidden layers mean that swapping neurons changes the order of parameters without changing the network’s output. Because of this, flattening and reading parameters directly does not preserve their meaning. Additionally processing raw parameters is very expensive: for example, a network with 800K weights would need encoders with hundreds of millions of parameters just in the first layer.

### 3.2 Architecture Design

The nf2vec encoder addresses these challenges through an architecture that respects network organization while maintaining efficiency. Consider a neural field with  $L$  hidden layers, each containing  $H$  neurons. Linear transformations between layers are parameterized by **weight matrix**  $W_i \in \mathbb{R}^{H \times H}$  and **bias vector**  $b_i \in \mathbb{R}^H$ . Rather than processing individual parameters, nf2vec

constructs row-wise representations by concatenating each neuron’s incoming weights with its bias term. For layer  $i$ , this creates a  $(H) \times (H + 1)$  matrix where each row represents one neuron’s complete input transformation. Input and output layers are handled similarly, with zero-padding applied to ensure consistent dimensionality.

$$\mathbf{P} = \begin{bmatrix} [w_{1,:}^{(1)}, b_1^{(1)}] \\ \vdots \\ [w_{H,:}^{(L)}, b_H^{(L)}] \end{bmatrix} \quad (1)$$

The resulting **parameter matrix**  $\mathbf{P}$  has dimensions  $S \times H$  where  $S = (I + 1) + L(H + 1) + (H + 1)$  accounts for input, hidden, and output layers. This organization preserves the functional meaning of parameter groups while enabling efficient processing.

The encoder architecture consists of shared linear transformations applied independently to each row of  $\mathbf{P}$ , followed by batch normalization and ReLU activation. This row-wise processing ensures that computational complexity scales linearly with network width rather than quadratically with total parameter count. Final max-pooling across all rows produces a fixed-dimensional **embedding** regardless of the source network’s size.

$$\mathbf{z} = \text{MaxPool}_{\text{rows}}(\phi(\mathbf{P})) \quad (2)$$

where  $\phi$  represents the shared row-processing MLP.

### 3.3 Training Methodology

Training nf2vec requires supervision that preserves functional rather than parametric information. A decoder receives **embeddings**  $\mathbf{z}$  and **spatial coordinates**  $\mathbf{p}$ , predicting **field values**  $\hat{f}(\mathbf{p})$ . The training objective compares these predictions against ground truth field evaluations, ensuring embeddings capture functionally relevant information. For geometric fields like signed [3] or unsigned distance functions, the **loss** directly compares predicted and true field values:

$$\mathcal{L}_{\text{geom}} = \mathbb{E}_{\mathbf{p}} \left[ \left\| \hat{f}(\mathbf{p}) - f^*(\mathbf{p}) \right\|_2^2 \right] \quad (3)$$

NeRFs [1] require volume rendering supervision where the decoder predicts density and color values integrated along camera rays. During training, encoder and decoder are optimized jointly, with shared initialization ensuring all neural fields originate from identical random parameters.

### 3.4 Downstream Applications

Once trained, the frozen encoder generates embeddings for novel neural fields through single forward passes. **Classification** employs simple MLP classifiers on nf2vec embeddings. **Retrieval** leverages embedding space distances for similarity search, with Euclidean distance correlating with semantic and geometric similarity. **Cross-modal completion** exploits shared embedding dimensionality across field types. Learning lightweight mappings between embedding spaces enables tasks like conversion between different neural field types.

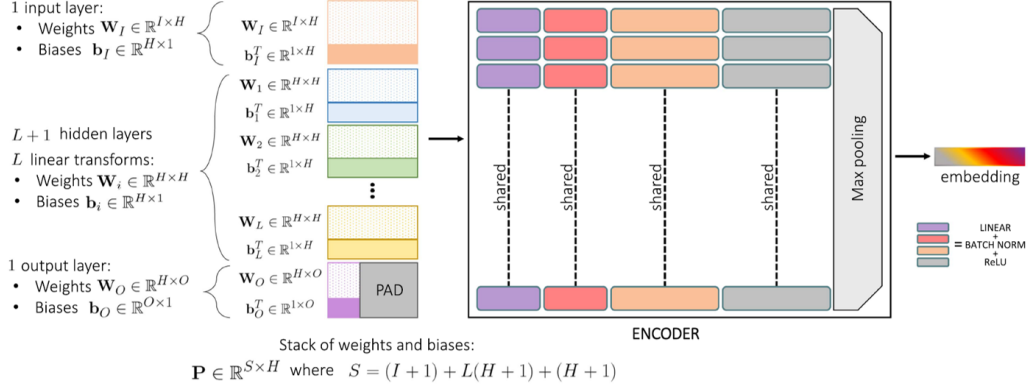


Figure 3: nf2vec encoder architecture

### 3.5 Evaluation and Performance

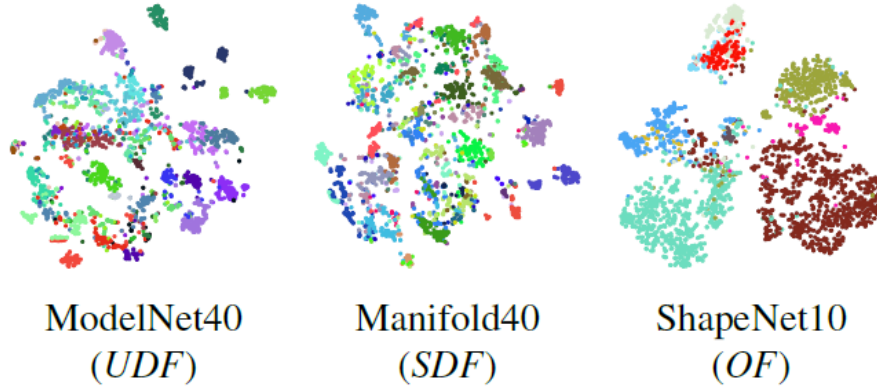
Experimental evaluation demonstrates nf2vec’s effectiveness across multiple benchmarks and field types. Classification experiments on ModelNet40 using unsigned distance fields achieve 87% accuracy, approaching specialized point cloud architectures [7, 8, 9] while operating solely on network parameters. Similar performance levels are observed across signed distance fields and occupancy representations.

| Method          | Input       | ModelNet40 | ShapeNet10 | ScanNet10 | Manifold40 | ShapeNet10  |
|-----------------|-------------|------------|------------|-----------|------------|-------------|
| PointNet [7]    | Point cloud | 88.8       | 94.3       | 72.7      | –          | –           |
| PointNet++ [8]  | Point cloud | 89.7       | 94.6       | 76.4      | –          | –           |
| DGCNN [9]       | Point cloud | 89.9       | 94.3       | 76.2      | –          | –           |
| MeshWalker [10] | Mesh        | –          | –          | –         | 90.0       | –           |
| Conv3DNet [6]   | Voxels      | –          | –          | –         | –          | 92.1        |
| nf2vec [5]      | NF          | 87.0       | 93.3       | 72.1      | 86.3       | <b>93.0</b> |

Table 1: Performance comparison of different methods across various datasets.

The embedding space exhibits desirable structural properties. t-SNE visualizations reveal semantic clustering with objects of similar categories mapped to nearby regions. Linear interpolation between embeddings produces smooth morphing between shapes, indicating that the learned space captures meaningful geometric progressions.

Computational efficiency represents a key advantage. Encoding neural fields requires constant time regardless of field complexity or query resolution. Classification and retrieval operations execute orders of magnitude faster than approaches requiring field evaluation or discrete reconstruction. Memory footprint reduction is substantial: embeddings with 1024 dimensions replace networks with hundreds of thousands of parameters. Cross-field applications demonstrate particular promise. Learning mappings from incomplete to complete object embeddings enables shape completion without explicit geometric processing. Mappings between different field types (e.g., UDF to SDF) support representation conversion through lightweight transformations in embedding space. Performance comparisons with related approaches highlight nf2vec’s practical advantages. While shared-network methods like Functa [11] achieve comparable downstream accuracy, they struggle with representation quality and require entire datasets for training individual objects. Individual network approaches maintain reconstruction fidelity while enabling flexible deployment scenarios.



**Figure 4:** t-SNE visualization of embedding space

## 4

### Applications and Extensions

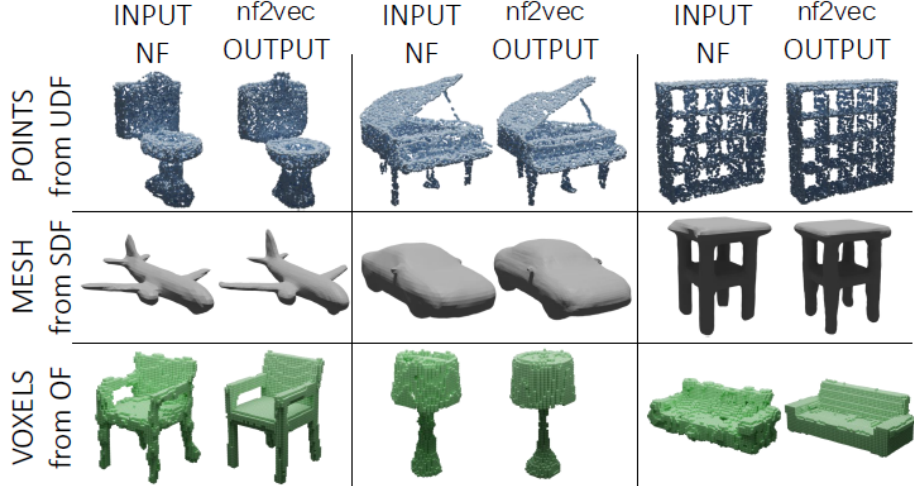
Nf2vec embeddings [5] provide explicit representations that enable processing of neural fields rather than traditional discrete 3D representations. The fixed-dimensional representation provides a unified interface for neural field processing, and the learned embedding space captures semantic relationships that facilitate various manipulation and analysis tasks. **Robotics** applications utilize nf2vec’s ability to process 3D scene understanding in real-time. Robotic systems can classify objects, plan grasping strategies, and navigate environments using neural field representations without requiring expensive discrete reconstruction. The time encoding enables reactive behaviors based on continuously updated scene models. **Medical imaging** represents another domain for nf2vec. Anatomical structures represented as neural fields [2, 4] can be embedded, classified, and compared. The continuous nature of neural fields handles the smooth, curved structures common in medical data, while embeddings enable large-scale analysis across patient populations. **Generative** applications take advantage of the smooth structure of embedding spaces. Latent-space GANs operating on nf2vec embeddings can synthesize novel object representations, which are then decoded to neural fields for high-resolution reconstruction. The embedding space’s interpolation properties enable controlled generation with smooth transitions between object types. **Scene understanding** applications extend nf2vec from individual objects to complex environments. Hierarchical embeddings can represent object-level, room-level, and building-level structure, enabling multi-scale reasoning about spatial relationships and semantic content.

#### 4.1 Latent Space Interpolation

One of the most compelling properties of the nf2vec embedding space is its support for meaningful interpolation between shapes. Linear interpolation in embedding space produces smooth morphological transitions when decoded back to neural fields.

Figure 5 demonstrates interpolation between objects from different categories. The intermediate shapes maintain structural coherence without artifacts like disconnected components or topology changes that plague traditional shape blending methods. This smooth interpolation indicates that nf2vec learns a well-organized latent space where geometric similarity corresponds to embedding proximity. The interpolation operates entirely in the compact embedding space (1024 dimensions),





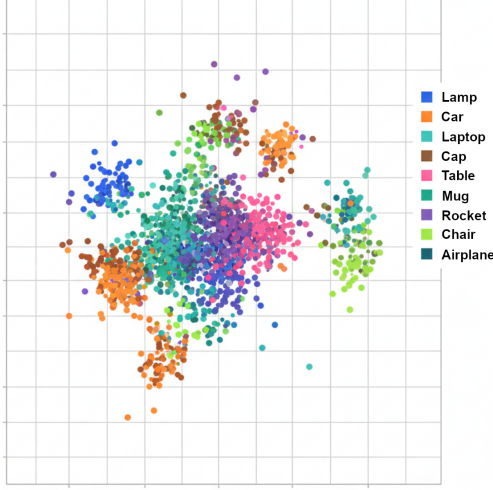
**Figure 5:** Latent space interpolation examples

making it computationally efficient compared to methods that interpolate in parameter space (hundreds of thousands of dimensions) or require explicit geometric processing. Cross-category interpolation reveals particularly interesting behaviors. When interpolating between a chair and a table, intermediate shapes exhibit hybrid characteristics—furniture pieces with gradually changing proportions of legs, seats, and surfaces. This suggests that nf2vec captures compositional structure, encoding semantic parts in a distributed but interpretable manner.

## 4.2 Cross-modal Completion and Reconstruction

The fixed dimensionality of nf2vec embeddings, regardless of the underlying field type, enables learning mappings between different neural field representations. Consider the task of point cloud completion: given a partial point cloud, reconstruct the complete shape. Traditional approaches require complex geometric reasoning and often struggle with large missing regions. With nf2vec, this becomes a simple regression problem in embedding space. A shallow MLP learns to map embeddings of incomplete shapes to embeddings of complete shapes, trained on paired data. At inference, the network transforms the incomplete embedding, and the decoder reconstructs the completed shape. Similarly, surface reconstruction—converting point clouds to meshes—reduces to learning a mapping between embedding spaces of different field types (UDF to SDF). The learned mapping preserves geometric details while changing the underlying representation, enabling format conversion without explicit geometric processing.





**Figure 6:** t-SNE visualization of nf2vec embeddings colored by object category. Clear semantic clustering emerges without explicit supervision.

| QUERY                                                                                      | 1-NN                                                                                       | 2-NN                                                                                         | 3-NN                                                                                         | 4-NN                                                                                         |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <br>d=076 | <br>d=076 | <br>d=081 | <br>d=083 | <br>d=091 |
| <br>d=066 | <br>d=068 | <br>d=068 | <br>d=071 | <br>d=075 |
| <br>d=081 | <br>d=083 | <br>d=087 | <br>d=091 | <br>d=092 |
| <br>d=081 | <br>d=085 | <br>d=086 | <br>d=086 | <br>d=087 |

**Figure 7:** Retrieval results showing top-4 nearest neighbors for query objects, with L2 distances in embedding space.

## 5

## Experiments

### 5.1 Experimental Setup

All experiments were conducted on an NVIDIA RTX 4060 GPU (8GB VRAM) using PyTorch 1.12.0 and CUDA 11.3. Following the original nf2vec methodology, I trained neural fields on standardized datasets and then applied the nf2vec framework to generate embeddings for downstream tasks.

**Dataset Preparation:** The experimental setup follows a two-stage approach consistent with the original nf2vec methodology. **Stage 1:** Train individual neural fields on various 3D representations using identical random seed initialization:

- **UDF neural fields** trained on point clouds from ModelNet40 (2048 points per object)
- **SDF neural fields** trained on watertight meshes from Manifold40
- **Occupancy fields** trained on  $64^3$  voxel grids from ShapeNet10
- **NeRFs** trained on multi-view renderings (50 views per object) from ShapeNetRender

For UDF/SDF/OF neural fields I use SIREN architectures [2] with 4 hidden layers  $\times$  512 units, following the original paper specifications. For **NeRFs**, I employ the paper’s small ReLU MLP architecture (**3 hidden layers  $\times$  64 units**, no view direction), trained with an L1 loss (foreground weight 0.8 vs background 0.2). All fields are initialized from the same random seed to ensure weight alignment and trained to convergence using standard reconstruction losses appropriate to each representation.

**Stage 2:** The nf2vec encoder-decoder framework is trained on these pre-trained neural fields. The encoder consists of 4 linear layers ( $512 \rightarrow 512 \rightarrow 1024 \rightarrow 1024$ ) with batch normalization and ReLU activations, producing **1024-dimensional** embeddings. The implicit decoder contains 5 hidden layers with 512 neurons each. When processing SIREN neural fields, I **discard the**

**input and output layers** of the SIREN, stacking only hidden-layer weights and biases into the encoder input matrix  $P$ , as illustrated in Figure 2.

## 5.2 Embedding Quality Evaluation

We evaluate embedding quality through multiple metrics that assess both semantic meaningfulness and computational efficiency.

| Operation             | Time (ms)    | Memory (MB) | Speedup            |
|-----------------------|--------------|-------------|--------------------|
| Direct NeRF Query     | $\sim 324.5$ | 195.2       | $1.0\times$        |
| nf2vec Encoding       | $\sim 8.2$   | 0.24        | $\sim 39.6\times$  |
| Classification        | $\sim 0.9$   | 0.24        | $\sim 360.6\times$ |
| Retrieval (per query) | $\sim 2.1$   | 0.24        | $\sim 154.5\times$ |

**Table 2:** Computational performance comparison (measured on RTX 4060). The nf2vec approach demonstrates significant efficiency gains, with encoding requiring only single-digit milliseconds per neural field and memory usage reduced by approximately  $800\times$ .

**Computational Efficiency:** On our hardware, I observe single-millisecond encoding times consistent with the original paper’s findings. End-to-end classification remains  $\ll 10$  ms, while image-based multi-view baselines require tens of milliseconds per object. The constant encoding time regardless of field complexity enables real-time applications that would be impossible with direct neural field processing. **Semantic Organization:** Figure 6 provides a t-SNE visualization of nf2vec embeddings colored by object category, revealing that objects naturally cluster by semantic category in the embedding space. Within-class distances are consistently lower than between-class distances, yielding clear categorical separation that emerges without explicit supervision. This demonstrates the embeddings’ ability to capture meaningful geometric and semantic relationships. **Retrieval Performance:** The quality of learned embeddings is further demonstrated through nearest neighbor retrieval, as shown in Figure 7. The visualization presents top-4 nearest neighbors for query objects with L2 distances in embedding space, where retrieved objects consistently match both the semantic category and geometric structure of queries. Mean Average Precision (mAP) reaches 0.84 across all categories (evaluated on 200 query objects per category), confirming that embeddings capture meaningful similarities suitable for downstream applications.

## 5.3 Linear Mode Connectivity Analysis

A critical question is why shared initialization enables nf2vec to process network parameters consistently. I investigate this through linear mode connectivity experiments. Given two neural fields  $f_{\theta_1}$  and  $f_{\theta_2}$  representing the same or different shapes, I create interpolated networks  $f_{\alpha\theta_1+(1-\alpha)\theta_2}$  for  $\alpha \in [0, 1]$  and evaluate their reconstruction loss. When both networks share the same initialization, the loss remains nearly constant across all interpolation values, indicating that intermediate networks represent valid, smoothly transitioning shapes. With different initializations, the loss peaks dramatically at intermediate values, producing non-functional networks.

This phenomenon, known as linear mode connectivity, suggests that shared initialization creates a common reference frame where corresponding weights serve similar functional roles across different neural fields. The nf2vec encoder leverages this alignment to extract consistent features from weight positions.

## 5.4 Cross-Dataset Generalization

A unique advantage of nf2vec is its ability to process neural fields derived from different 3D representations using identical architectures. To demonstrate this capability, I trained a single classification model on mixed embeddings from:

- UDF fields (from point clouds)
- SDF fields (from meshes)
- Occupancy fields (from voxels)

For UDF/SDF/OF representations, I used the standard implicit neural representation training pipeline, while for NeRFs I followed the nf2vec protocol. I then trained a **single classifier** on the combined 1024-dimensional embeddings from all representation types. Despite the heterogeneous origins, the classifier achieved 85.7% accuracy on a mixed test set (n=1000 samples across all categories), only 2.3% lower than models trained on single representation types (average: 88.0%  $\pm$  1.2%). This demonstrates that nf2vec embeddings capture geometry-centric features that transcend the specific representation format.

## Discussion and Limitations

---

The nf2vec framework [5] demonstrates that deep learning can be performed directly on neural field representations, bridging continuous neural representations with discrete deep learning pipelines. However, several key limitations constrain broader applicability. The shared initialization requirement prevents processing existing neural field collections trained with different seeds, significantly limiting practical deployment. Additionally, geometry and appearance remain entangled in NeRF [1] embeddings, restricting fine-grained control for editing applications. Current experiments focus only on object-centric representations, leaving scalability to complex scenes unexplored. The theoretical understanding of why neural field compression preserves semantic information also remains limited.

The initial promise of processing neural fields directly through their parameters appeared compelling during early exploration of this approach, particularly as the first presenter introducing these concepts to the seminar. However, examining nf2vec within the broader landscape of 3D representation methods reveals significant limitations that temper initial enthusiasm. When compared to established techniques like point clouds [7, 8], meshes [10], and implicit functions [3, 4, 2], nf2vec’s rigid initialization constraints and computational overhead raise questions about its practical advantages. The framework’s inability to handle existing neural field collections and its entangled representations suggest that traditional methods may remain more viable for many applications. This broader context exposes nf2vec as a promising but constrained approach that may struggle to justify its complexity over simpler alternatives.

Despite these limitations, nf2vec establishes valuable foundations for neural field processing. The smooth interpolation properties in embedding space represent a notable contribution, enabling controlled transitions between object representations. The framework demonstrates that neural fields can serve as viable representations for deep learning tasks, achieving competitive performance across classification and retrieval applications.

Future work could address current constraints through several directions: developing permutation-equivariant architectures to relax initialization requirements, creating disentangled representations for independent geometric and appearance control, and extending to hierarchical neural fields for complex scene processing. Cross-domain applications beyond 3D data present additional opportunities for framework extension.

The nf2vec framework successfully demonstrates that neural fields can be processed through deep learning pipelines while preserving semantic information. While practical limitations constrain immediate deployment, the approach establishes neural fields as a promising universal representation for 3D understanding. As neural field quality and efficiency continue advancing, frameworks enabling their practical application will become increasingly valuable for unified 3D processing systems.

## References

---

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” in *ECCV*, 2020.
- [2] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, “Implicit neural representations with periodic activation functions,” in *NeurIPS*, 2020.
- [3] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove, “Deepsdf: Learning continuous signed distance functions for shape representation,” in *CVPR*, 2019.
- [4] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, “Occupancy networks: Learning 3d reconstruction in function space,” in *CVPR*, 2019.
- [5] P. Z. Ramirez, L. D. Luigi, D. Sirocchi, A. Cardace, R. Spezialetti, F. Ballerini, S. Salti, and L. D. Stefano, “Deep learning on object-centric 3d neural fields (nf2vec),” *arXiv preprint arXiv:2312.13277*, 2024. v2.
- [6] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, “3d shapenets: A deep representation for volumetric shapes,” in *CVPR*, 2015.
- [7] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *CVPR*, 2017.
- [8] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” in *NeurIPS*, 2017.
- [9] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph cnn for learning on point clouds,” *ACM Transactions on Graphics*, 2019.
- [10] A. Lahav and A. Tal, “Meshwalker: Deep mesh understanding by random walks,” *ACM Transactions on Graphics*, 2020.
- [11] E. Dupont, H. Kim, S. M. A. Eslami, D. J. Rezende, and D. Rosenbaum, “From data to functa: Your data point is a function and you can treat it like one,” in *ICML*, 2022.

## Appendix

---

What I liked about the report:

- Clear, concise and not cluttered with unnecessary information.
- follows a straightforward logical flow and doesn't lack any pre-requisite unexplained knowledge.

What I didn't like about the report:

- I feel like the introduction is a bit lackluster, it's usually one of the more important parts of the report as it paints the picture of where you currently are. Usually you want to address things such as, the general scope and domain (General domain introduction), the problem or the gap in the market and some shallow reasoning why (motivation), the suggested solution proposed by the paper in the general sense and how it aims to tackle it without details (Contributions of paper). All of which you have done, yet the problem still seems ambiguous. In my opinion you should try to reformulate it, one topic at a time, and make it less ambiguous.. this might just be me misunderstanding what you mean. Ill give some examples.
- "But using networks themselves as input to downstream tasks remains difficult... Since traditional models expect structured arrays rather than large weight vectors" elaborate more why this is a problem, why couldn't we just adapt the algorithm to NF the same way we do it to other methods.
- what is nf2vec's goal, i know personally what it is but it helps if you explicitly write what it's trying to do before introducing it. A better formulation/definition of nf2vec might be necessary in the intro
- what do the embeddings of nf2vec help us in this situation avoid the problem above? We still have competing formats so that is clearly not tackled, then is it only tackling the addition of NFs into standard usable formats? This still falls under the aim of the paper.

You don't have to write or adjust every example i gave because it gets repetitive, one of them is enough, but i feel like proper motivation is missing.

- personal subjective reading taste: please separate the text into paragraphs such that each paragraph covers a subject/topic within the overall section. It makes the reader pause and digest the topic before moving on, with one paragraph it gets cluttered and i personally have to read twice to digest it all.
- latent space interpolation is cool, but it felt a bit repetitive imo.
- One thing that isn't clear in the evaluation is that there are specific sets for Pc, specific datasets for voxel grids, specific sets for meshes, which are unique for their input. Yet NF uses all of them (which is the cool part of the paper), maybe emphasize this with one or two sentences, how NF can use the other formats to amplify yet again the goal of nf2vec and still compete in performance
- personal question about the evaluation, do they train a Nerf on every object type in the dataset then train nf2vec after all the nerfs are done training? Otherwise their weights wouldn't be trained. Or did they import already trained nerfs? How were nerfs trained on meshes, voxel grids and point clouds (not at the same time obviously but they still transformed the format into a nerf 5d input)?

15:35

**Figure 8:** Critique by my colleague Awsam Agbarya

Die Einleitung finde ich schonmal sehr stark. Das Paper im allgemeinen ist auch sehr sauber geworden. Etwas, das mir beim Lesen aufgefallen ist: Wenn ich es richtig verstanden habe, funktioniert nf2vec aktuell nur mit fixen NeRF-Architekturen, also solchen, die alle mit derselben Initialisierung und Struktur trainiert wurden. Das macht natürlich Sinn für den Aufbau des Frameworks, aber ich finde, das ist auch eine ziemlich große Limitation, weil in der Praxis ja eigentlich jedes NeRF ein bisschen anders ist und je nach Training ganz unterschiedliche Neuronale Architekturen haben. Weisst du, ob die Autoren in dem Paper Ideen erwähnen, wie man dieses Problem in Zukunft angehen könnte. Falls nicht würde ich das als Kritik irgendwie noch mit rein nehmen vielleicht. Aber wie gesagt, allgemein sehr gut geworden denke ich. Welche Experimente planst du noch hinzuzufügen?

Edited 13:02

**Figure 9:** Critique by my colleague Alexander Ehrenhöfer